

DT-Fusion Chat Module — Backend API Specification

Version: 2.0 (Simplified)

Date: 2026-04-29

Audience: Backend (Spring Boot) developers

Frontend env variable: `VITE_CHAT_MODE=real` + `VITE_API_BASE_URL=http://host:port`

Architecture Overview

Layer	Endpoints	Purpose
REST	3 endpoints	Data fetching + media upload
WebSocket (STOMP)	5 send + 3 subscribe	Real-time messaging, typing, presence, reactions, read receipts

Design Principle: REST is for reading data and uploading files. WebSocket handles all real-time writes.

No "Create Conversation" endpoint — The server auto-creates conversations when a user is onboarded.

Table of Contents

- Data Models
- REST API Endpoints (3 total)
- WebSocket / STOMP Protocol
- Realtime Event Types & Payloads
- Message Lifecycle Flow
- Authentication Notes
- Spring Boot Quick Reference

1. Data Models

1.1 User

Field	Type	Nullable	Description
id	string	No	Unique user identifier
name	string	No	Full display name
email	string	Yes	Email address
avatarUrl	string	Yes	URL to profile image
presence	enum	No	ONLINE AWAY OFFLINE
lastSeenAt	string	Yes	ISO 8601 timestamp

1.2 Message

Field	Type	Nullable	Description
id	string	No	Server-generated unique message ID
clientId	string	Yes	Client-generated correlation ID (for optimistic updates)
conversationId	string	No	Parent conversation ID
senderId	string	No	User ID of the sender
body	string	No	Message text content
attachments	Attachment[]	No	Array of attachments (can be empty [])
reactions	Reaction[]	No	Array of reactions (can be empty [])
status	enum	No	PENDING SENT DELIVERED READ FAILED
createdAt	string	No	ISO 8601
updatedAt	string	No	ISO 8601
replyToId	string	Yes	ID of the message being replied to
editedAt	string	Yes	ISO 8601 if edited
deletedAt	string	Yes	ISO 8601 if soft-deleted
deliveredTo	string[]	No	User IDs the message was delivered to
readBy	string[]	No	User IDs who have read the message

1.3 Attachment

Field	Type	Nullable	Description
id	string	No	Unique attachment ID
kind	enum	No	IMAGE FILE AUDIO VIDEO
url	string	No	Download/display URL
name	string	No	Original file name
size	number	No	File size in bytes
mimeType	string	No	MIME type (e.g. image/png)
thumbnailUrl	string	Yes	Thumbnail URL for image/video previews

1.4 Reaction

Field	Type	Nullable	Description
emoji	string	No	Emoji character (e.g. 👍)
userIds	string[]	No	User IDs who reacted with this

2. REST API Endpoints

Base URL: Value of `VITE_API_BASE_URL` (e.g. `http://localhost:8080`)

Auth: All endpoints require `Authorization: Bearer <JWT>` header.

2.1 GET /api/chat/contacts — Sidebar Listing

Returns all direct contacts and groups for the authenticated user. This is the **only** endpoint needed to render the entire sidebar.

Response: `200 OK`

```
[
  {
    "conversationId": "c_1",
    "type": "DIRECT",
    "name": "Alice Carter",
    "avatarUrl": "https://cdn.example.com/alice.jpg",
    "presence": "ONLINE",
    "lastSeenAt": "2026-04-29T06:00:00Z",
    "lastMessage": {
      "body": "Sounds good! 👍",
      "senderId": "u_alice",
      "createdAt": "2026-04-29T05:50:00Z"
    },
    "unreadCount": 2,
    "participantIds": ["u_me", "u_alice"],
    "isMuted": false,
    "isPinned": false,
    "updatedAt": "2026-04-29T05:50:00Z"
  },
  {
    "conversationId": "c_3",
    "type": "GROUP",
    "name": "Launch Squad",
    "avatarUrl": null,
    "presence": null,
    "lastSeenAt": null,
    "lastMessage": {
      "body": "Thanks team!",
      "senderId": "u_me",
      "createdAt": "2026-04-28T08:00:00Z"
    },
    "unreadCount": 0,
    "participantIds": ["u_me", "u_dana", "u_chris", "u_alice"],
    "isMuted": false,
    "isPinned": true,
    "updatedAt": "2026-04-28T08:00:00Z"
  }
]
```

Response Fields:

Field	Type	Description
<code>conversationId</code>	<code>string</code>	Unique conversation ID
<code>type</code>	<code>enum</code>	<code>DIRECT</code> or <code>GROUP</code>
<code>name</code>	<code>string</code>	DIRECT → other user's full name. GROUP → group title
<code>avatarUrl</code>	<code>string?</code>	DIRECT → other user's avatar. GROUP → group avatar
<code>presence</code>	<code>enum?</code>	DIRECT only: <code>ONLINE</code> / <code>AWAY</code> / <code>OFFLINE</code> . null for GROUP
<code>lastSeenAt</code>	<code>string?</code>	DIRECT only: ISO 8601. null for GROUP
<code>lastMessage</code>	<code>object?</code>	Most recent message preview (body, senderId, createdAt)
<code>unreadCount</code>	<code>integer</code>	Unread count for the authenticated user
<code>participantIds</code>	<code>string[]</code>	All participant user IDs
<code>isMuted</code>	<code>boolean?</code>	Whether notifications are muted
<code>isPinned</code>	<code>boolean?</code>	Whether pinned to top of list
<code>updatedAt</code>	<code>string</code>	ISO 8601 — used for sorting (descending)

Notes:

- Sorted by `updatedAt` descending
- Server auto-creates conversations at user onboarding — no create endpoint needed
- The `name` field should be the **other user's full name** for DIRECT, and the **group title** for GROUP

2.2 GET /api/chat/messages/{conversationId} — Conversation Messages

Returns paginated messages for a selected conversation, along with **full participant details** (name, avatar, presence, lastSeen). This means the frontend needs **zero extra API calls** to render the chat header and messages.

Path Variable: `conversationId` (string)

Query Parameters:

Param	Type	Default	Description
<code>page</code>	integer	<code>1</code>	Page number (1-indexed)
<code>pageSize</code>	integer	<code>50</code>	Messages per page

Response: `200 OK`

```
{
  "data": [
    {
      "id": "m_1_1",
      "clientId": null,
      "conversationId": "c_1",
      "senderId": "u_alice",
      "body": "Hey! Are we still on for the design review?",
      "attachments": [],
      "reactions": [],
      "status": "READ",
      "createdAt": "2026-04-29T05:00:00Z",
      "updatedAt": "2026-04-29T05:00:00Z",
      "replyToId": null,
      "deliveredTo": ["u_me"],
      "readBy": ["u_me"]
    }
  ],
  "page": 1,
  "pageSize": 50,
  "total": 127,
  "hasMore": true,
  "participants": [
    {
      "id": "u_alice",
      "name": "Alice Carter",
      "email": "alice@example.com",
      "avatarUrl": "https://cdn.example.com/alice.jpg",
      "presence": "ONLINE",
      "lastSeenAt": "2026-04-29T06:00:00Z"
    },
    {
      "id": "u_me",
      "name": "You",
      "avatarUrl": null,
      "presence": "ONLINE",
      "lastSeenAt": null
    }
  ]
}
```

Notes:

- Messages sorted by `createdAt` ascending
 - Page 1 = most recent messages. Scrolling up triggers page 2, 3, etc.
 - `participants` array contains **full User objects** for every participant in this conversation
 - The frontend uses `participants` to render the chat header (name, avatar, status) and sender names on messages
-

2.3 POST `/api/chat/media` — Upload Media Files

Uploads one or more media files and returns attachment objects with server-hosted URLs. The client then embeds these attachments in a WebSocket message via `/app/chat.send`.

Content-Type: `multipart/form-data`

Request:

```
POST /api/chat/media
Content-Type: multipart/form-data

files: [photo.jpg, document.pdf]
```

Form Field	Type	Required	Description
<code>files</code>	<code>File[]</code>	Yes	One or more files to upload

Response: `200 OK`

```
{
  "attachments": [
    {
      "id": "att_abc123",
      "kind": "IMAGE",
      "url": "https://cdn.example.com/uploads/photo.jpg",
      "name": "photo.jpg",
      "size": 1024000,
      "mimeType": "image/jpeg",
      "thumbnailUrl": "https://cdn.example.com/uploads/thumb-photo.jpg"
    },
    {
      "id": "att_def456",
      "kind": "FILE",
      "url": "https://cdn.example.com/uploads/document.pdf",
      "name": "document.pdf",
      "size": 256000,
      "mimeType": "application/pdf",
      "thumbnailUrl": null
    }
  ]
}
```

Usage Flow:

1. User selects files → Frontend calls `POST /api/chat/media`
2. Server stores files, returns attachment objects with URLs
3. Frontend sends message via WebSocket `/app/chat.send` with the attachments embedded:

```
{
  "conversationId": "c_1",
  "body": "Check out this photo!",
  "clientId": "cl_xyz",
  "attachments": [
    { "id": "att_abc123", "kind": "IMAGE", "url": "https://...", "name": "photo.jpg",
      "size": 1024000, "mimeType": "image/jpeg" }
  ]
}
```


3. WebSocket / STOMP Protocol

3.1 Connection

Property	Value
Protocol	STOMP over WebSocket (SockJS fallback)
Endpoint	<code>ws://{host}/ws</code> or <code>wss://{host}/ws</code>
Auth Header	<code>connectHeaders: { Authorization: "Bearer <JWT>" }</code>
Heartbeat	Incoming: 10000ms, Outgoing: 10000ms
Reconnect	Auto-reconnect with 5000ms delay

3.2 Client → Server (SEND)

Destination	Payload	Description
<code>/app/chat.send</code>	<code>SendMessageRequest</code>	Send a new message
<code>/app/chat.typing</code>	<code>{ conversationId, userId }</code>	Typing start/stop
<code>/app/chat.read</code>	<code>{ conversationId, upToMessageId }</code>	Mark messages read
<code>/app/chat.reaction</code>	<code>{ conversationId, messageId, emoji, userId }</code>	Toggle reaction
<code>/app/presence</code>	<code>{ userId, status, lastSeenAt }</code>	Update own presence

`/app/chat.send`

```
{
  "conversationId": "c_1",
  "body": "Hello!",
  "clientId": "cl_xyz789",
  "replyToId": null,
  "attachments": []
}
```

`/app/chat.typing`

```
{ "conversationId": "c_1", "userId": "u_me", "typing": true }
```

`/app/chat.read`

```
{ "conversationId": "c_1", "upToMessageId": "m_15" }
```

`/app/chat.reaction`

```
{ "conversationId": "c_1", "messageId": "m_10", "emoji": "👍", "userId": "u_me" }
```

`/app/presence`

```
{ "userId": "u_me", "status": "ONLINE", "lastSeenAt": "2026-04-29T06:00:00Z" }
```

3.3 Server → Client (SUBSCRIBE)

Destination	Description
<code>/user/queue/events</code>	Primary channel. All events fan into this queue
<code>/topic/messages/{convId}</code>	Optional per-conversation topic
<code>/topic/presence</code>	Global presence broadcasts

4. Realtime Event Types & Payloads

Every event uses this envelope:

```
{ "type": "EVENT_TYPE", "ts": "ISO-8601", "payload": { ... } }
```

MESSAGE_NEW — New incoming message

```
{ "type": "MESSAGE_NEW", "ts": "...", "payload": { /* full Message object */ } }
```

MESSAGE_ACK — Server acknowledges sent message (clientId → serverId)

```
{ "type": "MESSAGE_ACK", "ts": "...", "payload": { "clientId": "cl_xyz", "message": { /*  
  * Message */ } } }
```

MESSAGE_DELIVERED — Message delivered to recipients

```
{ "type": "MESSAGE_DELIVERED", "ts": "...", "payload": { "messageId": "m_1", "conversationId": "c_1", "userIds": ["u_alice"] } }
```

MESSAGE_READ — Messages read by a participant

```
{ "type": "MESSAGE_READ", "ts": "...", "payload": { "conversationId": "c_1", "userId": "u_alice", "upToMessageId": "m_5", "readAt": "..." } }
```

TYPING_START / TYPING_STOP

```
{ "type": "TYPING_START", "ts": "...", "payload": { "conversationId": "c_1", "userId": "u_alice" } }
```

PRESENCE_UPDATE

```
{ "type": "PRESENCE_UPDATE", "ts": "...", "payload": { "userId": "u_bob", "status": "AWAY", "lastSeenAt": "..." } }
```

REACTION_ADDED / REACTION_REMOVED

```
{ "type": "REACTION_ADDED", "ts": "...", "payload": { "conversationId": "c_1", "messageId": "m_1", "emoji": "👍", "userId": "u_alice" } }
```

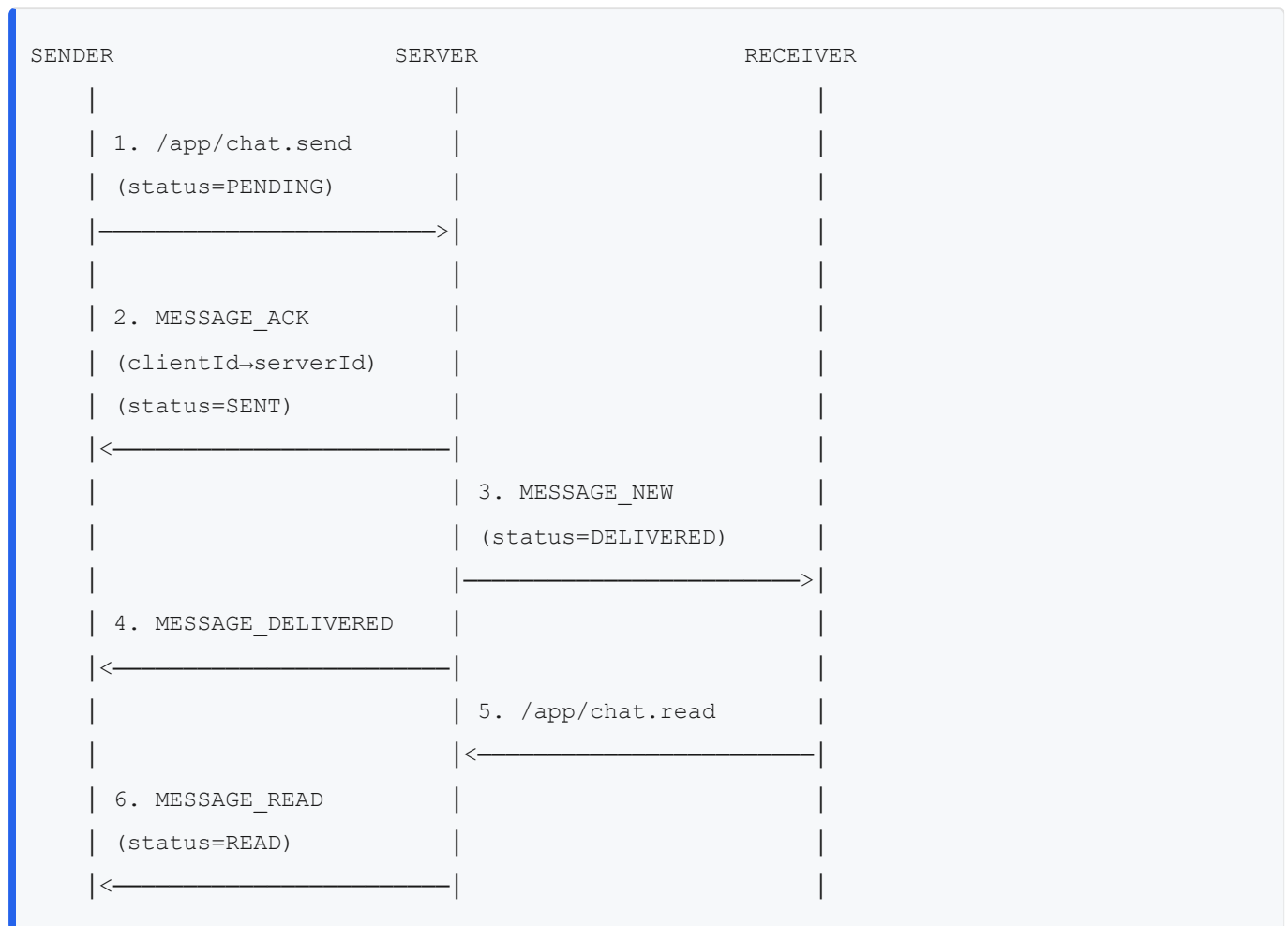
CONVERSATION_CREATED — Server auto-created a new conversation

```
{ "type": "CONVERSATION_CREATED", "ts": "...", "payload": { /* full Conversation object */ } }
```

UNREAD_COUNT_UPDATED

```
{ "type": "UNREAD_COUNT_UPDATED", "ts": "...", "payload": { "conversationId": "c_1", "unreadCount": 3 } }
```

5. Message Lifecycle Flow



Status flow: `PENDING → SENT → DELIVERED → READ`

Error path: `PENDING → FAILED` (retry when WebSocket reconnects)

6. Authentication Notes

- All REST endpoints require `Authorization: Bearer <JWT>` header
- WebSocket sends JWT in STOMP `connectHeaders`
- Server determines `currentUserId` from the JWT — never trust client-provided `senderId`
- `/user/queue/events` is automatically scoped to the authenticated user

7. Spring Boot Quick Reference

```
// REST Controller – 3 endpoints only

@RestController
@RequestMapping("/api/chat")
public class ChatController {

    @GetMapping("/contacts") // §2.1 – Sidebar listing
    public List<ContactItemDto> getContacts() { }

    @GetMapping("/messages/{conversationId}") // §2.2 – Paginated messages
    public ConversationMessagesResponse getMessages(
        @PathVariable String conversationId,
        @RequestParam(defaultValue = "1") int page,
        @RequestParam(defaultValue = "50") int pageSize
    ) { }

    @PostMapping("/media") // §2.3 – Media upload
    public MediaUploadResponse uploadMedia(
        @RequestParam("files") MultipartFile[] files
    ) { }
}

// WebSocket Controller
@Controller
public class ChatWebSocketController {

    @MessageMapping("/chat.send") // Send message
    @MessageMapping("/chat.typing") // Typing indicator
    @MessageMapping("/chat.read") // Mark read
    @MessageMapping("/chat.reaction") // Toggle reaction
    @MessageMapping("/presence") // Presence update
}

// WebSocket Config
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    // Endpoint: /ws (with SockJS fallback)
    // App destination prefix: /app
    // Broker prefixes: /topic, /queue
}
```